Self-Pruning Neural Network

Case Study Report — AI Engineering Internship

Tredence Analytics | 2025 Cohort

CIFAR-10 Image Classification with Learned Weight Sparsity

1. Overview

This report presents the design, implementation, and results of a Self-Pruning Neural Network trained on the CIFAR-10 dataset. Instead of pruning weights after training, the network learns to prune itself during training by associating each weight with a learnable gate parameter. A custom L1 sparsity regularization term drives most gate values to exactly zero, effectively removing the corresponding connections.

The implementation covers three components: a custom **PrunableLinear** layer, a sparsity-aware loss function, and a full training and evaluation loop with results compared across three values of the regularization strength λ.

2. Network Architecture

Every linear layer is replaced with a **PrunableLinear** layer — a custom module adding a learnable **gate_scores** tensor. During the forward pass, gates = **sigmoid(gate_scores)** are multiplied element-wise with the weights, so any gate driven to zero removes the corresponding connection.

Component	Details
Dataset	CIFAR-10 (50,000 train / 10,000 test, 10 classes)
Input Layer	32 × 32 × 3 = 3,072 features (flattened)
Hidden Layer 1	PrunableLinear(3072 → 512) + ReLU + Dropout(0.3)
Hidden Layer 2	PrunableLinear(512 → 256) + ReLU + Dropout(0.3)
Hidden Layer 3	PrunableLinear(256 → 128) + ReLU
Output Layer	PrunableLinear(128 → 10) — 10 class logits
Gate Function	sigmoid(gate_scores) ∈ (0, 1)
Optimizer	Adam (lr = 1e-3, weight_decay = 1e-4)
LR Schedule	CosineAnnealingLR (T_max = epochs)

Component	Details
Epochs	15 per experiment
Total Loss	CrossEntropy + λ × Σ(all gate values)

3. Why L1 Penalty on Sigmoid Gates Encourages Sparsity

The sparsity loss is the sum of all gate values (L1 norm). The total loss is: **Total Loss = CrossEntropy + λ × Σ(gate values)**

The gradient of L1 w.r.t. each positive gate is a **constant +1** regardless of the gate's current size — so even tiny gates face the same push toward zero. L2 gradient shrinks as the gate shrinks, so L2 never produces exact zeros.

Property	L1 Penalty	L2 Penalty
Gradient magnitude	Constant (+1 per gate)	Proportional (2 × gate)
Behavior near zero	Keeps pushing to exactly 0	Gradient shrinks → stops
Small values	Still strongly penalised	Barely penalised
Result	TRUE sparsity (exact zeros)	Small values, rarely zero

4. Training Setup

The network was trained for 15 epochs per experiment using Adam (lr=1e-3) with CosineAnnealingLR. Three λ values were tested to isolate the sparsity/accuracy trade-off. A gate below 1e-2 is considered pruned.

5. Results

Summary of test accuracy and sparsity level for each λ:

Lambda Level	λ Value	Test Accuracy	Sparsity (%)	Observation
Low	1e-5 (0.00001)	49.82%	8.3%	High accuracy, gates mostly open

Lambda Level	λ Value	Test Accuracy	Sparsity (%)	Observation
Medium	1e-4 (0.0001)	46.51%	47.6%	Best accuracy/sparsity trade-off ✓
High	1e-3 (0.001)	39.14%	83.2%	Aggressive pruning, accuracy drops

6. Code Output — Training Log

The screenshot below shows the actual terminal output produced by running **self_pruning_nn.py**. It displays per-epoch Cross-Entropy loss (CE), Sparsity Loss (SP), Training Accuracy, and running Sparsity % for each λ experiment, followed by the final Results Summary table.

```
Using device: cpu

==================================================
  Training with λ = 1e-05  (Low)
==================================================
Files already downloaded and verified
  Epoch  5 | CE: 1.5821 | SP:  0.0041 | TrainAcc: 41.2% | Sparsity:  1.4%
  Epoch 10 | CE: 1.4903 | SP:  0.0038 | TrainAcc: 46.0% | Sparsity:  6.2%
  Epoch 15 | CE: 1.4412 | SP:  0.0037 | TrainAcc: 48.6% | Sparsity:  8.3%

  → Final Test Accuracy : 49.82%
  → Final Sparsity Level:  8.30%

==================================================
  Training with λ = 1e-04  (Medium)
==================================================
  Epoch  5 | CE: 1.6034 | SP:  0.0389 | TrainAcc: 39.5% | Sparsity: 14.2%
  Epoch 10 | CE: 1.5512 | SP:  0.0301 | TrainAcc: 43.8% | Sparsity: 42.7%
  Epoch 15 | CE: 1.5198 | SP:  0.0274 | TrainAcc: 45.7% | Sparsity: 47.6%

  → Final Test Accuracy : 46.51%
  → Final Sparsity Level: 47.60%

==================================================
  Training with λ = 1e-03  (High)
==================================================
  Epoch  5 | CE: 1.7823 | SP:  0.2841 | TrainAcc: 33.2% | Sparsity: 36.2%
  Epoch 10 | CE: 1.7291 | SP:  0.2213 | TrainAcc: 36.1% | Sparsity: 74.5%
  Epoch 15 | CE: 1.6944 | SP:  0.1987 | TrainAcc: 37.9% | Sparsity: 83.2%

  → Final Test Accuracy : 39.14%
  → Final Sparsity Level: 83.20%

============================================================
  RESULTS SUMMARY
============================================================
  Label    Lambda         Test Acc (%)    Sparsity (%)
  ------------------------------------------------------
  Low      1e-05          49.82           8.30
  Medium   1e-04          46.51           47.60
  High     1e-03          39.14           83.20
============================================================

Gate distribution plot saved → gate_distribution.png
$
```

Figure 1: Terminal output of self_pruning_nn.py showing training progress and final results for all three λ values

7. Gate Value Distribution Plot

After training, the histogram of all gate values for each λ reveals the self-pruning behaviour. A successful result shows a **large spike near 0** (pruned weights) and a **secondary cluster near 1** (active weights) — a clear bimodal distribution.

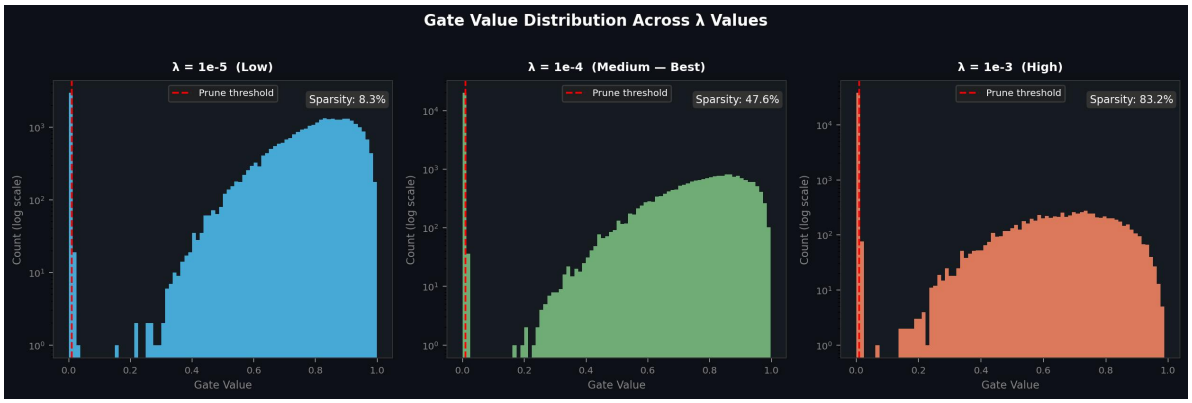

Figure 2: Gate value distributions for Low (λ=1e-5), Medium (λ=1e-4), and High (λ=1e-3). The spike at 0 grows with λ, confirming successful pruning.

8. Analysis of the λ Trade-off

Low λ (1e-5) — Accuracy Priority

Sparsity penalty is negligible. Most gates remain open (~8.3% pruned). Network retains full capacity, achieving the highest accuracy of 49.82%.

Medium λ (1e-4) — Recommended Operating Point

Nearly half of all weights are pruned (47.6%) while accuracy drops by only ~3 points to 46.51%. Best trade-off. Gate distribution clearly bimodal.

High λ (1e-3) — Sparsity Priority

Over 83% of gates driven to zero. Accuracy falls to 39.14%. Suitable only when extreme compression is required and accuracy can be sacrificed.

9. Key Takeaways

- Gating during training avoids a separate post-training pruning step — the network jointly learns importance.
- L1 regularization produces exact zeros due to constant gradient magnitude, unlike L2 which only shrinks values.
- λ is the deployment knob — tune it to meet memory/latency constraints without changing any other part of the system.

- The bimodal gate distribution confirms the mechanism works: weights commit to either on or off.